

Przedmiot: Przegląd metod i narzędzi AI Temat: Praca domowa 2 – Analiza danych zbioru Heart Disease oraz model regresji liniowej (grupa B)	Prowadzący: Kajetan Zarzycki
Wykonał: <i>Marcin Zubowski, numer PS: 2, numer indeksu: 119084</i> <i>Kierunek Data Science, semestr II</i>	Data: 25.04.2026

Spis treści

1. Wstęp	1
2. Charakterystyka zbioru danych	1
a. Opis	2
b. Rozkład zmiennych	4
3. Jak działa algorytm Regresji Logistycznej?	5
4. Omówienie kodu	5
5. Eksperymenty i wizualizacje	7
a. Wagi cech	7
b. Granica decyzyjna	8
c. Macierz pomyłek i krzywa ROC	9
6. Wnioski	11

1. Wstęp

Celem zadania jest analiza zbioru Heart Disease, przeprowadzenie eksperymentów, wykonanie wizualizacji oraz stworzenie i wykorzystanie modelu regresji logistycznej, który pozwoli sklasyfikować pacjentów pod kątem obecności choroby serca na podstawie ich wyników klinicznych. Projekt przechodzi przez proces przygotowania i czyszczenia danych, analizę zbioru, eksperymenty z parametrami oraz ewaluację wyników za pomocą metryk czy wykresów. Zadanie wykonano w Pythonie przy użyciu takich bibliotek jak: sklearn, pandas, numpy czy matplotlib.

2. Charakterystyka zbioru danych

a. Opis

Zbiór Heart Disease został zaimportowany za pomocą biblioteki ucimlrepo (UCI Machine Learning Repository). Do eksperymentów wykorzystano najczęściej stosowaną bazę pochodzącą z kliniki w Cleveland. Zbiór oryginalnie składa się 303 rekordów pacjentów oraz 14 atrybutów (kolumn). Jest on bardzo zróżnicowany, zawiera zarówno dane ciągłe, katagoryczne czy binarne:

- **age**: Wiek pacjenta wyrażony w latach (zmienna ciągła).
- **sex**: Płeć pacjenta (zmienna binarna: 1 = mężczyzna, 0 = kobieta).
- **cp**: Typ odczuwanego bólu w klatce piersiowej (zmienna katagoryczna: 1 = typowa dławica piersiowa, 2 = nietypowa dławica, 3 = ból niesercowy, 4 = bezobjawowy).
- **trestbps**: Spoczynkowe ciśnienie tętnicze krwi mierzone przy przyjęciu do szpitala w mm Hg (zmienna ciągła).
- **chol**: Poziom cholesterolu w surowicy krwi pacjenta wyrażony w mg/dl (zmienna ciągła).
- **fbs**: Poziom cukru we krwi na czczo (zmienna binarna: 1 = wartość powyżej 120 mg/dl, co może sugerować cukrzycę; 0 = wartość w normie).
- **restecg**: Wynik spoczynkowego badania elektrokardiograficznego (zmienna katagoryczna: 0 = w normie, 1 = nieprawidłowości załamka ST-T, 2 = przerost lewej komory serca).
- **thalach**: Maksymalne tętno osiągnięte przez pacjenta podczas testu wysiłkowego (zmienna ciągła).
- **exang**: Dławica piersiowa wywołana wysiłkiem fizycznym (zmienna binarna: 1 = tak, 0 = nie).
- **oldpeak**: Obniżenie odcinka ST na wykresie EKG wywołane wysiłkiem w stosunku do stanu spoczynkowego, będące wskaźnikiem niedokrwienia (zmienna ciągła zmiennoprzecinkowa).
- **slope**: Nachylenie szczytowego odcinka ST podczas maksymalnego wysiłku (zmienna katagoryczna: 1 = rosnące, 2 = płaskie, 3 = opadające).
- **ca**: Liczba głównych naczyń krwionośnych zabarwionych podczas badania fluoroskopowego, brak zabarwienia świadczy o zwężeniu (zmienna porządkowa: wartości od 0 do 3).
- **thal**: Wynik badania w kierunku talasemii, czyli dziedzicznej wady krwi (zmienna katagoryczna: 3 = prawidłowy przepływ, 6 = defekt utrwalony, 7 = defekt odwracalny).

- **num:** Zmienna docelowa. Diagnoza obecności choroby serca oraz jej stopień zaawansowania (zmienna kategoryczna: 0 = zdrowy, wartości 1-4: chory w danym stopniu).

Zbiór w 6 rekordach posiadał brakujące wartości (dla atrybutów ca i thal).

```
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         303 non-null    int64
1   sex         303 non-null    int64
2   cp          303 non-null    int64
3   trestbps    303 non-null    int64
4   chol        303 non-null    int64
5   fbs         303 non-null    int64
6   restecg     303 non-null    int64
7   thalach     303 non-null    int64
8   exang       303 non-null    int64
9   oldpeak     303 non-null    float64
10  slope       303 non-null    int64
11  ca          299 non-null    float64
12  thal        301 non-null    float64
13  num         303 non-null    int64
```

```
=== Missing Values ===
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  num
87   53   0   3      128   216    0         2     115     0      0.0    1  0.0  NaN   0
166  52   1   3      138   223    0         0     169     0      0.0    1  NaN   3.0   0
192  43   1   4      132   247    1         2     143     1      0.1    2  NaN   7.0   1
266  52   1   4      128   204    1         0     156     1      1.0    2  0.0  NaN   2
287  58   1   2      125   220    0         0     144     0      0.4    2  NaN   7.0   0
302  38   1   3      138   175    0         0     173     0      0.0    1  NaN   3.0   0
```

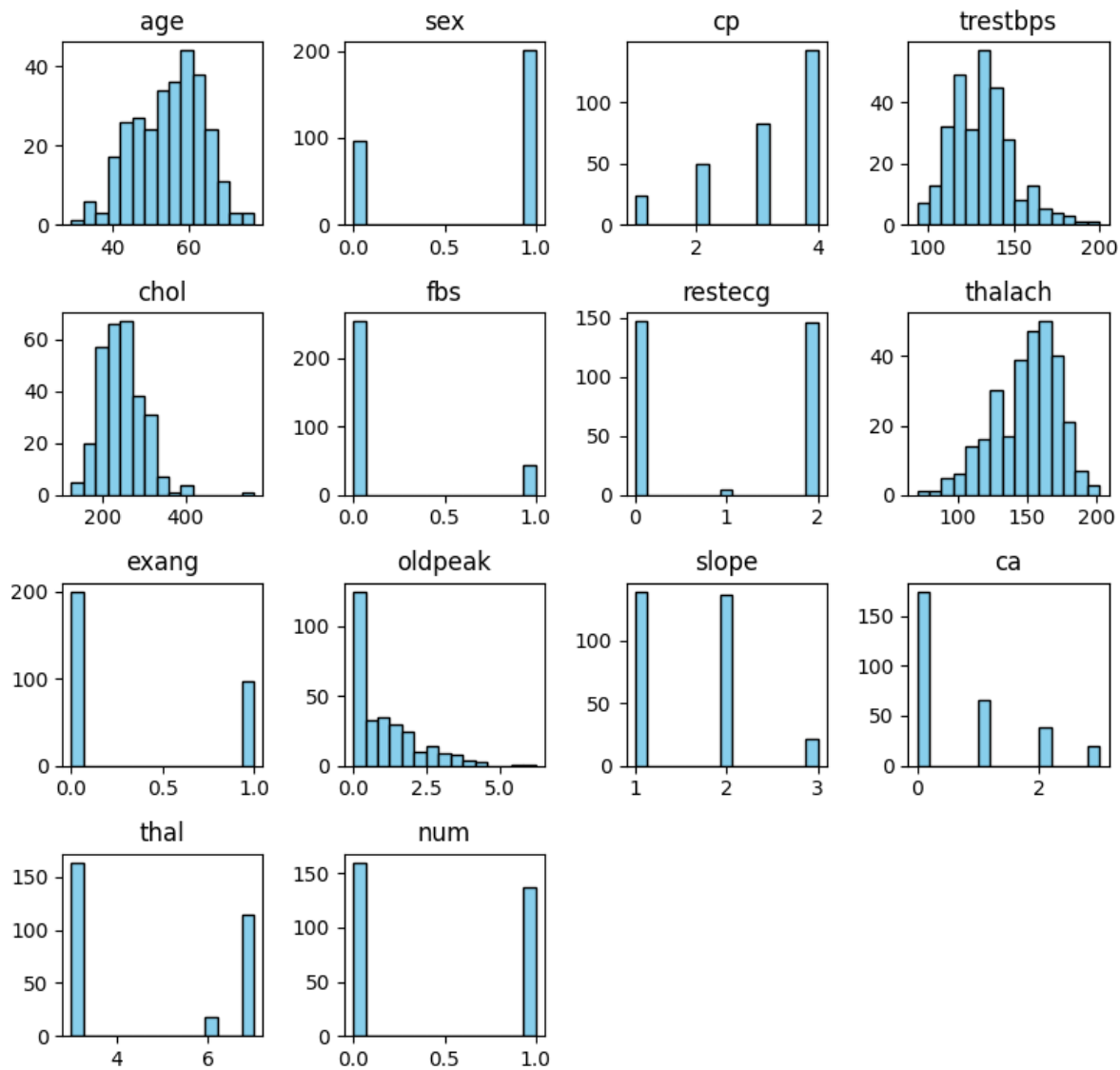
Ze względu na to, że zbiór zawiera jedynie 6 wierszy z brakami (ok. 2% całego zbioru) podjęto decyzję o ich usunięciu ze zbioru (funkcja `dropna()`), aby nie fałszować danych medycznych. Dodatkowo przekształcono zmienną docelową, która pierwotnie przyjmowała wartości od 0 do 4 (stopień zaawansowania), na postać binarną tworząc problem klasyfikacji pacjentów na chorych/zdrowych. Po oczyszczeniu zbioru i oddzielenia od niego zmiennej docelowej (w celu późniejszego trenowania modelu) jego kształt wyglądał następująco.

```
Dataset shape after cleaning: (297, 13), Target distribution:
num
0    160
1    137
```

Dodatkowo widać, że zbiór jest niemal równy pod względem zdrowych – 160 osób (~54%) i chorych – 137 osób (~46%).

b. Rozkład zmiennych

Histogramy przedstawiające rozkład poszczególnych zmiennych:



Oraz ich statystyki (średnia, mediana, minimum, maksimum):

```
age - Mean: 54.54, Median: 56.00, Min: 29.00, Max: 77.00
sex - Mean: 0.68, Median: 1.00, Min: 0.00, Max: 1.00
cp - Mean: 3.16, Median: 3.00, Min: 1.00, Max: 4.00
trestbps - Mean: 131.69, Median: 130.00, Min: 94.00, Max: 200.00
chol - Mean: 247.35, Median: 243.00, Min: 126.00, Max: 564.00
fbs - Mean: 0.14, Median: 0.00, Min: 0.00, Max: 1.00
restecg - Mean: 1.00, Median: 1.00, Min: 0.00, Max: 2.00
thalach - Mean: 149.60, Median: 153.00, Min: 71.00, Max: 202.00
exang - Mean: 0.33, Median: 0.00, Min: 0.00, Max: 1.00
oldpeak - Mean: 1.06, Median: 0.80, Min: 0.00, Max: 6.20
slope - Mean: 1.60, Median: 2.00, Min: 1.00, Max: 3.00
ca - Mean: 0.68, Median: 0.00, Min: 0.00, Max: 3.00
thal - Mean: 4.73, Median: 3.00, Min: 3.00, Max: 7.00
```

3. Jak działa algorytm Regresji Logistycznej?

Zanim przejdziemy do implementacji i omówienia modelu regresji logistycznej warto krótko wytłumaczyć jak mniej więcej ona działa. Wbrew nazwie „regresja” jest to algorytm często służący do klasyfikacji. Każda cecha otrzymuje pewną wagę, która jest przepuszczana przez specjalną funkcję logistyczną (Sigmoida):

$$S(x) = \frac{1}{1 + e^{-x}}$$

Funkcja ta zwraca dla dowolnej liczby wartość z przedziału od 0 do 1. Dzięki temu model zwraca prawdopodobieństwo przynależności do danej klasy. Jeśli wartość jest powyżej ustalonego progu (domyślnie 0.5), to algorytm klasyfikuje pacjenta jako chorego (1), w przeciwnym przypadku jako zdrowego (0).

4. Omówienie kodu

```
def train_logistic_regression(X, y, selected_features=None):
    if selected_features is not None:
        X = X[selected_features].values
    else:
        X = X.values

    X_train, X_test, y_train, y_test = train_test_split(X, y.values, test_size=0.2, random_state=42)
```

Proces budowy modelu odbywa się w funkcji „train_logistic_regression” (plik logistic_regression_model.py). Funkcja przyjmuje zbiór atrybutów/cech (X), zmienną docelową (y) oraz opcjonalną listę wybranych kolumn (selected_features - pozwala to wybrać dla na jakich cechach zostanie stworzony model co pozwoli na testowanie różnych kombinacji, które mogą osiągać lepsze/gorsze wyniki od modelu przetrenowanego na pełnym zbiorze. Dodatkowo jest to wykorzystywane przy wizualizacji jednego z późniejszych eksperymentów). W pierwszej kolejności algorytm sprawdza, czy został podany argument „selected_features”. Jeśli tak, zbiór danych jest filtrowany. Następnie zbiór zostaje podzielony na zbiór treningowy (80% pacjentów), na którym model będzie się uczył i zbiór testowy (20% pacjentów – „test_size=0.2”) służący do oceny

końcowej niewidzianych wcześniej wyników. Dzięki parametrowi „random_state=42” gwarantowana jest powtarzalność wyników.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Zbiór medyczny posiada drastycznie różne skale (np. cholesterol sięgający aż 300 mg/dl oraz oldpeak oscylujący w okolicach 1). Właśnie dlatego wykorzystano obiekt StandardScaler z biblioteki sklearn, który przekształca rozkład każdej cechy. Skaler został dopasowany (fit_transform) wyłącznie do danych treningowych. Dopiero wyuczone na nich parametry posłużyły do przeskalowania danych testowych (transform), co całkowicie wyeliminowało ryzyko wycieku danych (data_leakage).

```
parameters = {"C": [0.001, 0.01, 0.1, 1, 10, 100]}

grid_search = GridSearchCV(LogisticRegression(random_state=42), parameters, cv=5)
grid_search.fit(X_train_scaled, y_train)
model = grid_search.best_estimator_
```

Następnie wykonywana jest optymalizacja parametru C odpowiadającego za odwrotność siły regularyzacji. Niska wartość – silna regularyzacja co może doprowadzić do niedouczenia (Underfitting – obniżona dokładność dla danych treningowych i testowych), wysoka wartość – słaba regularyzacja ryzyko przeuczenia (Overfitting – model uczy się danych na pamięć przez co dokładność będzie sztucznie wysoka na danych treningowych, ale dla danych testowych poradzi sobie okropnie). Właśnie dlatego szukamy idealnego C dla naszego modelu przy pomocy GridSearchCV, które automatycznie trenuje i ocenia modele dla podanych wartości ostatecznie wybierając ten, gdzie nie doszło ani do przeuczenia, ani niedouczenia. Parametr „cv=k” odpowiada za k-krotną walidację krzyżową.

```
y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print(f"Logistic Regression Accuracy: {accuracy:.2f}")
print(f"Best C Parameter: {grid_search.best_params_['C']}")
return model, scaler, X_train_scaled, X_test_scaled, y_train, y_test
```

Na koniec algorytm oblicza dokładność modelu i wypisuje najlepszy parametr C. Funkcja oprócz samego modelu zwraca również skaler oraz podzielone zbiory co przydaje się innym funkcjom do tworzenia raportów czy wizualizacji bez

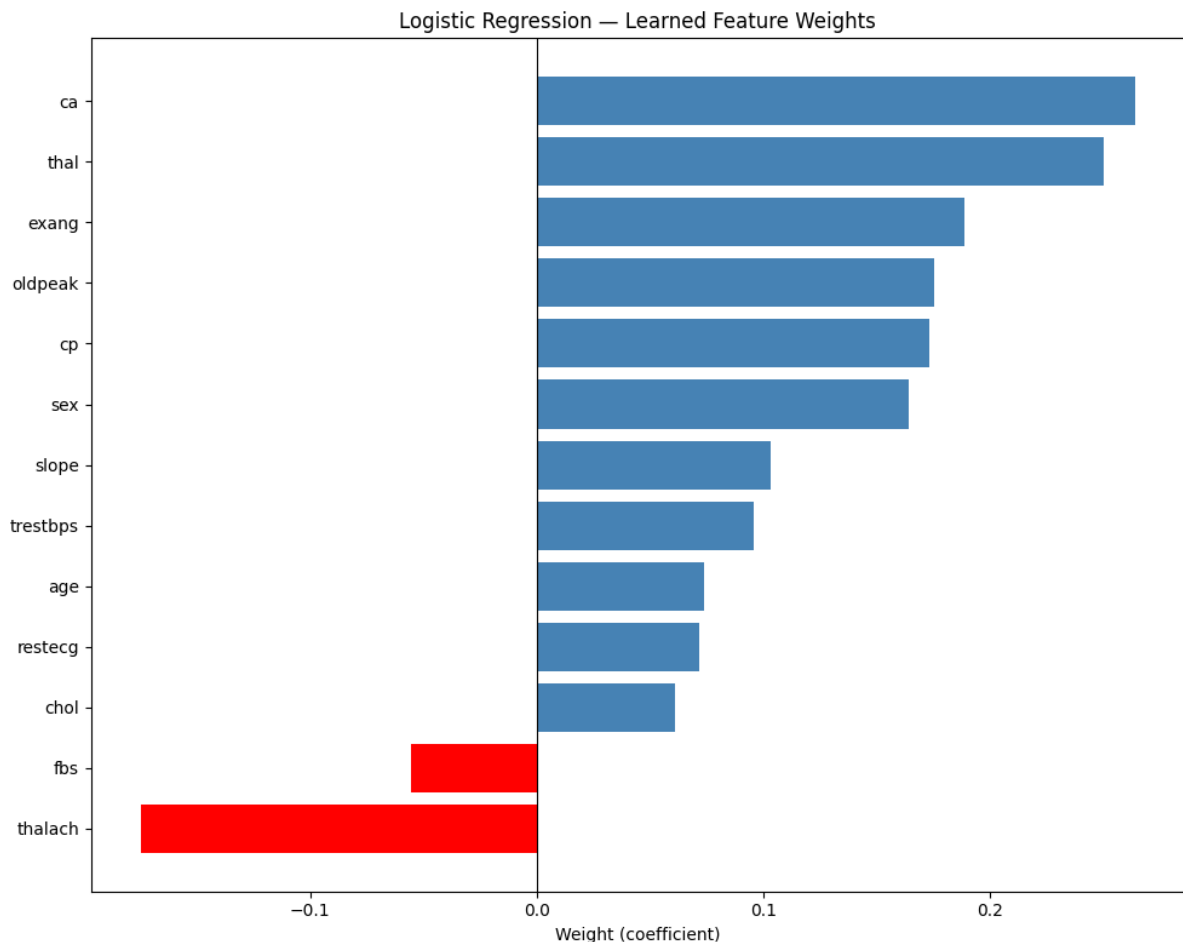
konieczności ponownego liczenia. Dla zbioru wszystkich cech otrzymujemy następujące wyniki:

```
Logistic Regression Accuracy: 0.88  
Best C Parameter: 0.01
```

5. Eksperymenty i wizualizacje

a. Wagi cech

Po stworzeniu modelu sprawdzamy, jak wyglądają wyuczone współczynniki wagowe. Wyniki dodatnie otrzymują cechy, które zwiększają prawdopodobieństwo zakwalifikowania pacjenta do grupy chorych (im wyższa wartość tym silniejsza waga). Wyniki ujemne to cechy, które zwiększają szansę na przypisanie do osób zdrowych (im niższa wartość tym silniejsza waga, lecz w stronę zdrowych). Tak oto wyglądają wagi naszego modelu:

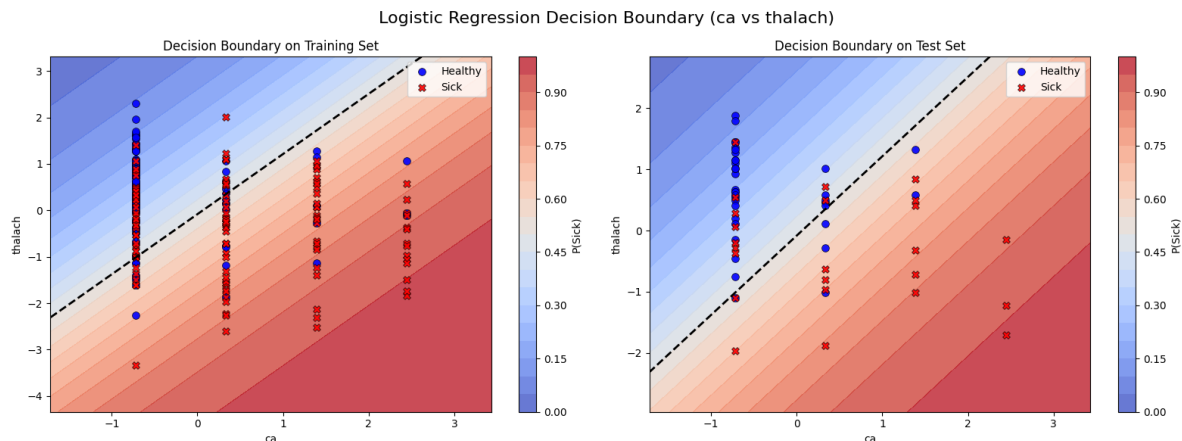


Z powyższego wykresu wynika, że do czynników najbardziej zwiększających ryzyko diagnozy choroby (największe wagi na osi) należą: ca i thal. Z kolei czynnikami chroniącymi/zmniejszającymi to ryzyko jest thalach.

b. Granica decyzyjna

Aby zrozumieć, w jaki sposób model separuje przestrzeń pacjentów na zdrowych i chorych, przeprowadzono eksperyment polegający na celowej redukcji wymiarowości. Wytrenowano pomocnicze modele regresji logistycznej zasilane wyłącznie dwiema wybranymi cechami, co pozwoliło na czytelne rzutowanie wielowymiarowej przestrzeni decyzyjnej na płaszczyznę 2D.

Eksperyment 1: „ca” vs „thalach”

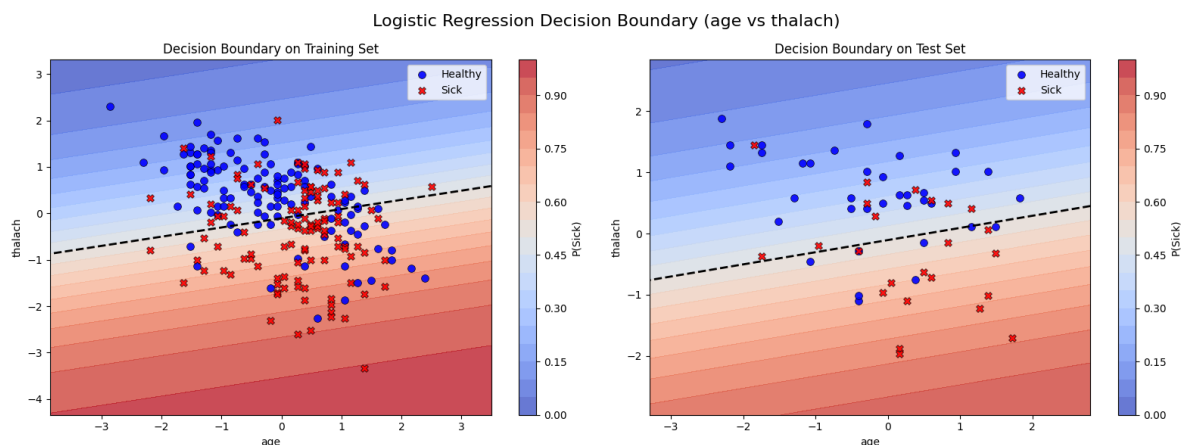


Na wykresie ułożyły się charakterystyczne pionowe kolumny ze względu na to, że cecha „ca” przyjmuje jedynie wartości 0, 1, 2 lub 3. Wykres wyraźnie udowadnia, że wraz ze skokiem wartości „ca” w prawą stronę, algorytm drastycznie podnosi prawdopodobieństwo choroby. Dodatkowo poniżej znajdują się wyniki, dla tego modelu

```
Logistic Regression Accuracy: 0.75  
Best C Parameter: 0.1
```

Eksperyment 2: „age” vs „thalach”

Tym razem wykorzystano zmienną ciągłą co pozwoli na uzyskanie „klasycznego” wykresu klasyfikacyjnego.



Przerywana linia wyznacza próg odcięcia algorytmu, w którym prawdopodobieństwo wynosi równo 0.5. Dodatkowo na wykresach widnieje mapa ciepła, która obrazuje strefy wysokiej i niskiej pewności modelu (im dalej od granicy tym model jest pewniejszy). Wyniki tego modelu:

```
Logistic Regression Accuracy: 0.70  
Best C Parameter: 1
```

Eksperymenty te pokazują nam również spadek dokładności dla mniejszej ilości i mniej dokładnych cech.

c. Macierz pomyłek i krzywa ROC

Końcowym etapem jest weryfikacja skuteczności modelu.

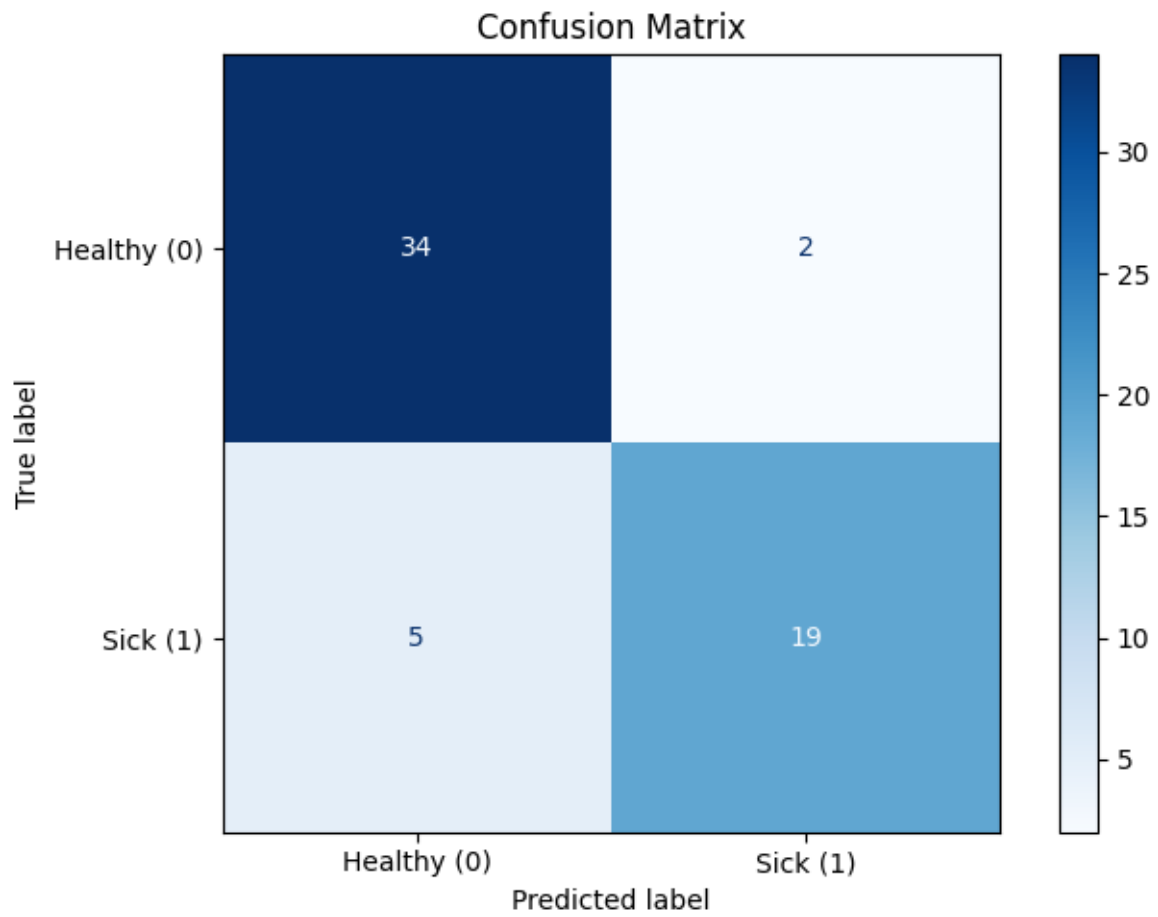
- **Raport klasyfikacyjny**

Wykorzystano narzędzie „classification_report” z biblioteki „sklearn.metrics” do oceny precyzji (Precision), czułości (Recall) oraz miary F1 dla obu klas. W medycznym zastosowaniu modelu klasyfikacyjnego priorytetem jest optymalizacja parametru Recall dla klasy 1, co przekłada się na minimalizowanie ryzyka nierozpoznanie choroby u pacjentów faktycznie obciążonych.

```
=== Classification Report ===  
              precision    recall  f1-score   support  
  
Healthy (0)       0.87      0.94      0.91        36  
Sick (1)          0.90      0.79      0.84        24  
  
accuracy          0.88  
macro avg         0.89      0.87      0.88        60  
weighted avg      0.88      0.88      0.88        60
```

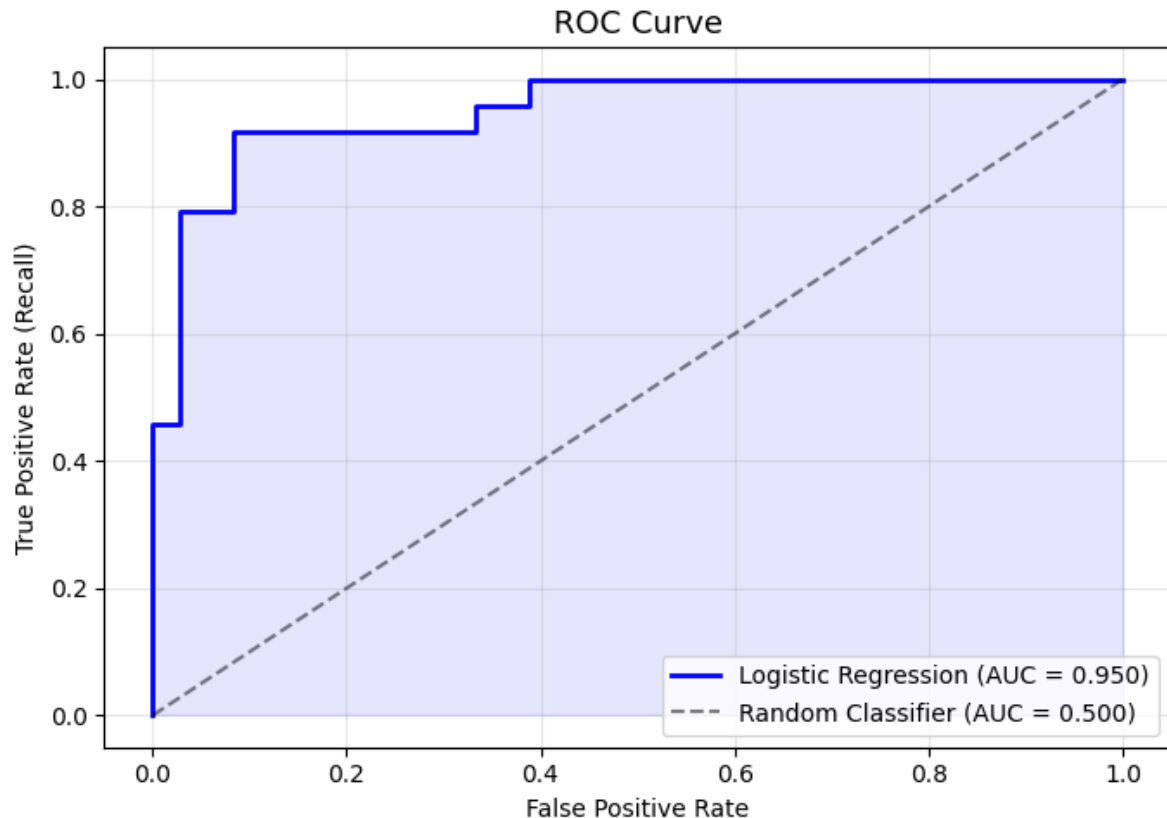
- **Macierz pomyłek**

Ukazuje dokładny rozkład trafnych i błędnych diagnoz modelu na zbiorze testowym. Zestawia ze sobą liczby przypadków prawdziwie pozytywnych (TP) i prawdziwie negatywnych (TN) wraz z fałszywymi alarmami oraz krytycznymi z punktu widzenia zdrowia pacjenta przeoczeniami choroby.



- **Krzywa ROC**

Wykres ukazuje znakomity balans między wskaźnikiem prawdziwie pozytywnych (TPR) a fałszywie pozytywnych (FPR) diagnoz. Wynik AUC (Area Under the Curve) jest na bardzo wysokim poziomie, co świadczy o znacznej przewadze predykcyjnej modelu nad losowym zgadywaniem.



6. Wnioski

- **Wysoka skuteczność diagnostyczna:** Zastosowanie algorytmu Regresji Logistycznej na zbiorze badawczym Heart Disease przyniosło bardzo satysfakcjonujące rezultaty analityczne. Model osiągnął dokładność na poziomie 88% na zbiorze testowym. W uwarunkowaniach medycznych, gdzie niemożliwym jest uchwycenie pełnej złożoności biologicznej pacjenta jest to wynik wysoce realistyczny i dowodzący dużej siły predykcyjnej wybranych 13 parametrów klinicznych. Parametr regularyzacji na poziomie $C = 0.01$ skutecznie uchronił model przed przeuczeniem.
- **Analiza błędów medycznych:** Macierz pomyłek dostarczyła kluczowych wniosków. Na 60 pacjentów ze zbioru testowego model poprawnie sklasyfikował 53 osoby, popełniając zaledwie 7 błędów (2 fałszywe alarmy u osób zdrowych i 5 nierozpoznanych przypadków chorobowych). Przekłada się to na bardzo wysoki wskaźnik Recall dla pacjentów zdrowych (0.94) oraz precyzję na poziomie 0.90 przy wykrywaniu chorych.
- **Krzywa ROC:** Osiągnięta wartość pola pod krzywą na poziomie $AUC = 0.950$ świadczy o dobrym balansie i niemal idealnej separacji klas, znacznie przewyższając granicę losowego zgadywania.